

OBERON

Product Strategy, Technical Blueprint
and Commercialization Plan

THE DEFINITIVE PRODUCT DIRECTION

DOCUMENT PURPOSE

A definitive product and company blueprint

This report consolidates the AI-centered Product Brief, the Market Strategy and Scale Plan, and the comprehensive research analysis into one operating document. It is written to serve two audiences: the founder making build and market decisions, and external partners or investors evaluating whether Oberon solves a real problem.

Field	Current conclusion
Product stage	Idea stage - pre-MVP
Recommended product	Open, self-hostable Earth observation monitoring and evidence engine
First wedge	Vegetation disturbance and restoration monitoring
Primary paying customer	Environmental and geospatial consultancies, followed by restoration and carbon project operators
Business model	Apache 2.0 core + paid pilots + managed monitoring + task products + enterprise
Demand status	Category demand proven; demand for Oberon specifically remains unvalidated
Go decision	Proceed as a staged validation program, not as a broad platform build

The core decision

Oberon should exist only as a focused operational workflow. It should not be positioned as a generic satellite API, an arbitrary Earth question-answering system, or a solution to satellite downlink constraints.

How to read this report

Sections 1-5 define what the product is and why the problem matters. Sections 6-9 explain the technical and commercial system. Sections 10-13 define execution, scaling, risks, and the pitch. All costs and probabilities are planning estimates and must be updated after benchmarking and customer discovery.

. Contents

Section	Topic	Purpose
1	Executive decision	What to build, what not to build, and why now.
2	The actual bottleneck	Where the Earth observation workflow still breaks.
3	Definitive product definition	The product, users, outputs, and boundaries.
4	Market demand and customers	Who pays, which segments to avoid, and real prospects.
5	AI role and trust model	Where AI adds value and how it must be evaluated.
6	Technical architecture	MVP design, scaling path, and deployment profiles.
7	Competitive positioning and moat	How Oberon differs and what can become defensible.
8	Open-source and commercial boundary	What is free, what is paid, and why.
9	Go-to-market and revenue	Pilots, pricing, sales motion, and conversion.
10	Economics and operational scale	Costs, unit economics, and capacity decisions.
11	Roadmap and team	24-month plan, hiring triggers, and capital strategy.
12	Risks, decision gates, and scenarios	When to continue, pivot, or stop.
13	External pitch summary	A concise product and investor narrative.
Appendix	Sources and validation notes	Primary sources and claim discipline.

Research discipline

A market can be real while a specific product is still unvalidated. This report separates category evidence, technical feasibility, and Oberon-specific demand instead of treating adjacent success as proof.

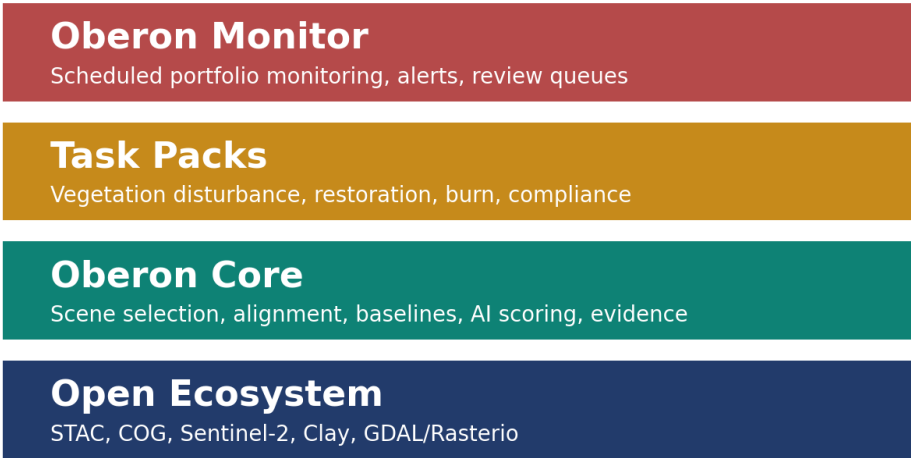
1. Executive Decision

The product is viable as a focused monitoring workflow, not as a broad Earth intelligence platform.

The verdict

Oberon has a credible reason to exist if it owns the last-mile decision workflow between public Earth observation data and operational action. The strongest first product continuously compares defined land portfolios, ranks likely material changes, and returns enough evidence for a person or downstream system to verify the finding.

What the product should become



Open source builds trust and adoption. Paid value lives in operation, specialization and accountability.

Figure 1. The recommended product stack and commercial boundary.

What Oberon should be

Definitive product statement
Oberon is an open, self-hostable Earth observation monitoring engine that turns public satellite observations into ranked, evidence-backed change findings for repeated land portfolios.

What Oberon should not be

- A generic "ask the Earth anything" API. Arbitrary questions are difficult to evaluate and easy to overclaim.
- A raw imagery marketplace. Planet, SkyFi, Copernicus, AWS and others already address imagery access.
- A claim that the Earth observation ecosystem has no standards or open-source tools. STAC, COG, eoAPI, TiTiler, openEO and Raster Vision already solve major layers.
- A defense-ready or air-gapped system at MVP. Disconnected operation, compliance and secure procurement are separate products.
- A single-binary Rust purity project. Rust is valuable in the control plane, but Python is the practical model and geospatial runtime for the first release.

Viability scorecard

Dimension	Assessment	Reason
Technical feasibility	Medium-high	The data, standards, libraries and model backbone exist. Integration and evaluation remain nontrivial.
Category demand	High	Organizations already buy remote monitoring, GeoAI, imagery and analyst-supported evidence.
Exact product demand	Unvalidated	The market may prefer managed dashboards

		or services over a self-hosted API.
AI necessity	Conditional	AI is useful only if it improves ranking quality or analyst efficiency over simple baselines.
Solo-founder buildability	Medium	Possible with one sensor, one task, batch-first deployment and strict scope.
Near-term revenue	Medium	Paid pilots and applied implementation are plausible before self-serve SaaS.
Long-term scale	Unknown	Requires a repeatable vertical, proprietary evaluation data and retained operational use.

Go recommendation

Proceed for 90 days with a technical benchmark and structured customer discovery. Do not invest in a broad hosted platform until one customer pays for a repeated workflow.

2. The Actual Bottleneck

The scarce resource is trusted analyst attention, not raw imagery.



The scarce resource is not imagery. It is trusted analyst attention and repeatable decision workflow.

Figure 2. The stack is increasingly standardized upstream; decision workflow remains fragmented.

The original framing was too far upstream

Satellite downlink, onboard compute and ground-station capacity are real industry constraints, but they are not the buying problem Oberon solves. Oberon operates after imagery has been downlinked, processed and exposed through public catalogs. A customer considering Oberon is choosing between manual GIS work, analyst services, hosted monitoring products, or custom scripts - not between orbital compute architectures.

The real last-mile failure

Earth observation data is available, but operational use still requires a chain of brittle decisions: selecting comparable observations, rejecting cloud and shadow, aligning dates and pixels, calculating interpretable measurements, running a model, ranking findings, preserving provenance, and routing uncertainty into human review. Each component exists independently. The product opportunity is the complete, repeatable workflow.

Layer	What already exists	What remains valuable
Discovery	STAC catalogs and public metadata	Selecting the best before/after evidence for a specific workflow
Raster access	COGs, TiTiler, eoAPI and cloud-native reads	Localized extraction, quality masks and reproducible artifacts
Model primitives	Clay and other geospatial foundation models	Task evaluation, ranking, calibration and abstention
Training tools	Raster Vision and Python geospatial ML libraries	A ready-to-query runtime and evidence contract
Commercial products	Lens, Picterra, LGND, Planet and vertical vendors	Open deployment, programmable workflow and customer-controlled models
Human operation	GIS analysts, consultants and field inspections	Reducing review burden without removing human accountability

Why this needs solving

- Repeated monitoring does not scale linearly with human review. Hundreds or thousands of polygons create a triage problem.
- Satellite imagery is not automatically comparable across dates. Clouds, shadows, seasonality, sensor differences and registration errors create false change.
- Operational users need geometry, ranking, evidence and auditability, not just an image or an embedding vector.
- The highest-value output is often a prioritized review queue: where should a human look first, and why?

The defensible gap

Too few products combine public imagery, open models, deterministic checks, provenance and self-hostable deployment into one review-ready monitoring workflow.

3. Definitive Product Definition

One complete workflow before a broad platform.

Initial user job

Job to be done

Show me where this land portfolio materially changed between two periods, rank the findings, explain the evidence, and let my team verify or dismiss each result.

The first product: Oberon Vegetation Monitor

The first commercialized workflow should monitor vegetation disturbance and restoration across defined polygons. It should detect likely loss or recovery, rank the most important regions, and provide before/after imagery, spectral evidence, change geometry, quality flags and model provenance.

Capability	MVP behavior	Reason
Portfolio input	Upload or register polygons and date windows	Matches repeated operational monitoring
Scene selection	Automatically rank locally valid observations	Removes manual catalog and cloud review
Evidence preparation	Read COG windows, mask invalid pixels and align dates	Creates analysis-ready comparable inputs
Baseline analytics	NDVI, NBR, pixel and quality deltas	Provides interpretable reference signals
AI triage	Clay feature difference and optional task head	Ranks complex change beyond a single index
Output	GeoJSON, imagery, overlay, evidence and review state	Supports APIs, reports and human verification
Uncertainty	Abstain when inputs or model evidence are weak	Prevents confident-looking failure

Recommended API contract

POST /v1/change

```
{
  "geometry": {"type": "Polygon", "coordinates": [...]},
  "before": {"from": "2026-04-01", "to": "2026-04-30"},
  "after": {"from": "2026-06-01", "to": "2026-06-30"},
  "task": "vegetation_disturbance",
  "max_cloud_fraction": 0.15
}
```

Response: ranked polygons + change score + spectral evidence +
quality state + before/after artifacts + model and source provenance

Product boundaries

Build now	Build after proof	Do not promise yet
Sentinel-2 L2A, one task, batch API, Docker Compose	Scheduled monitoring, review queue, alerts, one evaluated task head	Arbitrary natural language Earth answers
Before/after artifacts, baselines, Clay feature map	Managed cloud, team collaboration, customer feedback labels	Universal satellite protocol
Rust API + Python workers	Sentinel-1, commercial imagery adapters, enterprise controls	Production air-gap and defense readiness

Product discipline

The product should earn expansion. Reliability of one task comes before multiple sensors, a model marketplace, Kubernetes or an Oberon protocol.

4. Market Demand and Customer Priority

Demand is proven for monitoring; Oberon-specific demand is not.

Evidence that the category is real

Commercial and nonprofit organizations already pay for remote land monitoring, imagery, analyst services, model inference and compliance evidence. Lens reports broad adoption across conservation and other sectors. Picterra sells environmental intelligence and enterprise GeoAI workflows. LGND sells developer-oriented geospatial embeddings. Digital monitoring and remote sensing are increasingly embedded in forest carbon, restoration and deforestation compliance workflows.

Customer priority: budget vs. ability to win

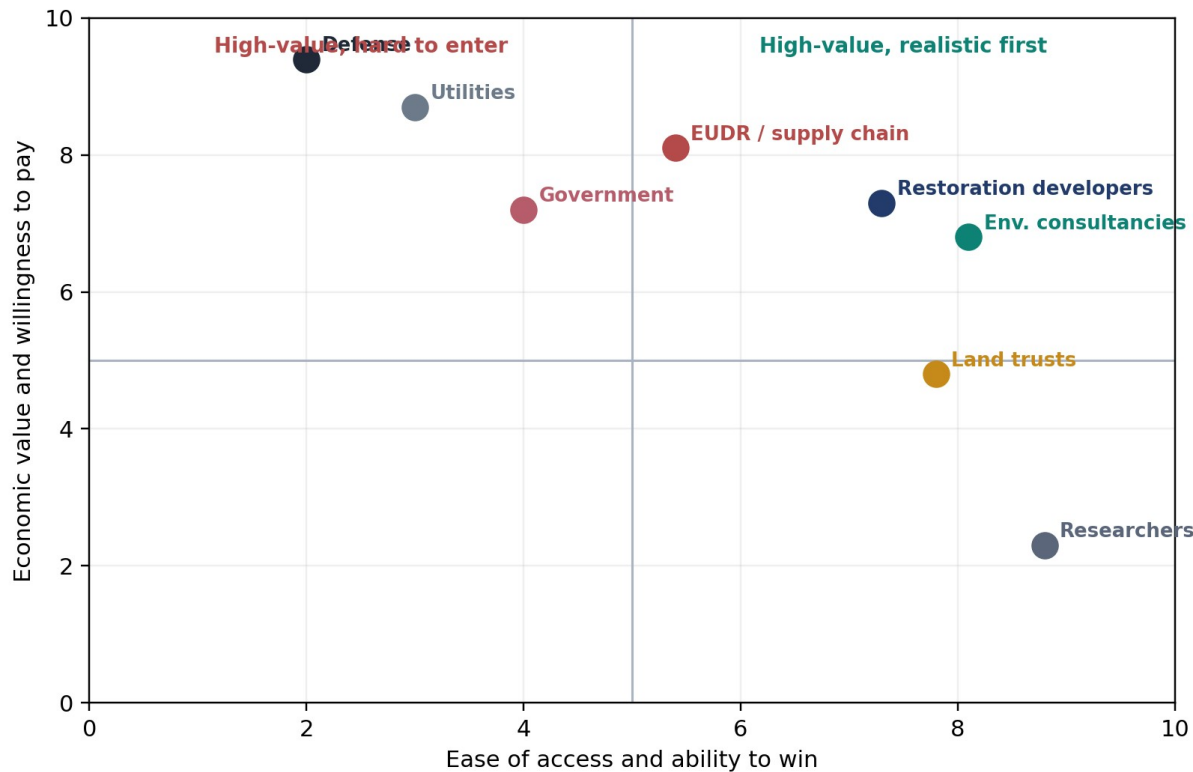


Figure 3. The first customer should balance budget with a realistic path to access and trust.

Recommended customer sequence

Priority	Customer type	Why they buy	How Oberon enters
1	Environmental and geospatial consultancies	Repeated preprocessing, analyst time and inconsistent provenance across client projects	White-label or internal engine, paid historical benchmark, reusable reporting
2	Restoration, forest and carbon project developers	Long-lived portfolios, MRV, funder evidence and field-prioritization	Vegetation disturbance and recovery monitoring pilot
3	Conservation networks and land trusts	Stewardship monitoring, annual review and limited field capacity	Design partnership, lower-cost public data workflow, case study
Later	EUDR, utilities, government and defense	Higher budgets and stronger compliance consequences	Enter only after accuracy, integrations and procurement capability mature

Named organizations to qualify

The following organizations are research-based prospect examples, not confirmed leads or statements of buying intent. Each must be qualified for current tools, budget, portfolio size and willingness to pilot.

Organization or group	Likely role	Recommended approach
Osa Conservation	Design partner	Benchmark one restoration corridor in exchange for workflow feedback and a publishable technical case.

Mast Reforestation	Commercial pilot candidate	Backtest post-wildfire recovery and disturbance on a limited project area.
Blue Forest	Commercial or research pilot	Lead with portfolio evidence, analyst time and reproducibility.
Forest Carbon Works / Chestnut Carbon	High-value, harder access	Compare Oberon against an existing monitoring workflow on non-sensitive areas.
Regional land trusts and conservation districts	Design partner or small paid pilot	Target teams with GIS staff, repeated parcel review and no locked-in incumbent.
Terraformation / Terraware ecosystem	Partner or benchmark	Study integrated ground-plus-remote workflows; do not assume they are an easy buyer.

Interview questions that reveal demand

- How many areas do you monitor, and how often are they reviewed?
- What triggered the last imagery review, and what output did the team produce?
- Where does staff time go: scene selection, preprocessing, interpretation, reporting or field verification?
- What happens financially or operationally when a material change is missed?
- Which examples have already been reviewed and can support evaluation?
- Why would API integration or self-hosting matter compared with an existing dashboard?

Demand gate

If 15-20 serious interviews reveal curiosity but no repeated review workflow, do not build a monitoring SaaS. Keep Oberon as an open-source technical project.

5. Where AI Belongs

AI must reduce review burden, not merely decorate the pipeline.

The hybrid intelligence model

Layer	Purpose	Output
Scene quality	Reject unusable observations	Valid pixels, cloud/shadow, snow, date offset
Deterministic indices	Provide interpretable physical signals	NDVI, NBR, water and pixel deltas
Foundation model	Represent complex multispectral patterns	Feature maps, embeddings and change distance
Task head	Translate features into a defined domain output	Vegetation loss, restoration recovery or burn mask
Calibration	Make scores safe enough for decisions	Thresholds, precision/recall, probability calibration
Provenance	Make findings reproducible	Scene IDs, bands, model, config and artifacts

When AI creates real value

- The user monitors hundreds or thousands of locations and cannot inspect every new observation.
- The meaningful signal spans multiple bands or spatial patterns that a single index cannot represent.
- The product must rank likely changes so analysts review the highest-value cases first.
- A task-specific head can produce polygons or categories required by another system.
- Human review feedback can improve thresholds, region-specific calibration and later task models.

When AI should be removed

If a deterministic baseline such as NDVI change, NBR change or pixel difference matches or exceeds Clay on held-out customer portfolios, the default product should not pay the latency, deployment and trust cost of the model. Oberon can remain valuable as a workflow and evidence engine with optional AI modules.

Required benchmark

Question	Minimum release gate
Does AI beat simple methods?	Compare pixel difference and relevant spectral indices on held-out locations.
Does it improve ranking?	Measure precision at K and analyst time per 100 areas.
Does it generalize?	Use geographic and temporal holdouts, not random neighboring patches.
Can scores be trusted?	Calibrate task probabilities; do not label raw similarity as confidence.
When does it refuse?	Return abstention for clouds, seasonal mismatch, weak evidence and out-of-distribution inputs.
Can a user verify it?	Always return before/after imagery, overlay, measurements and model/source provenance.

AI release rule

Do not launch an AI claim based on a visually impressive demo. Publish baseline comparisons, held-out results, known limitations and failure examples.

6. Technical Architecture and Scalability

Batch-first, hybrid, observable and independently scalable.

Recommended scalable architecture

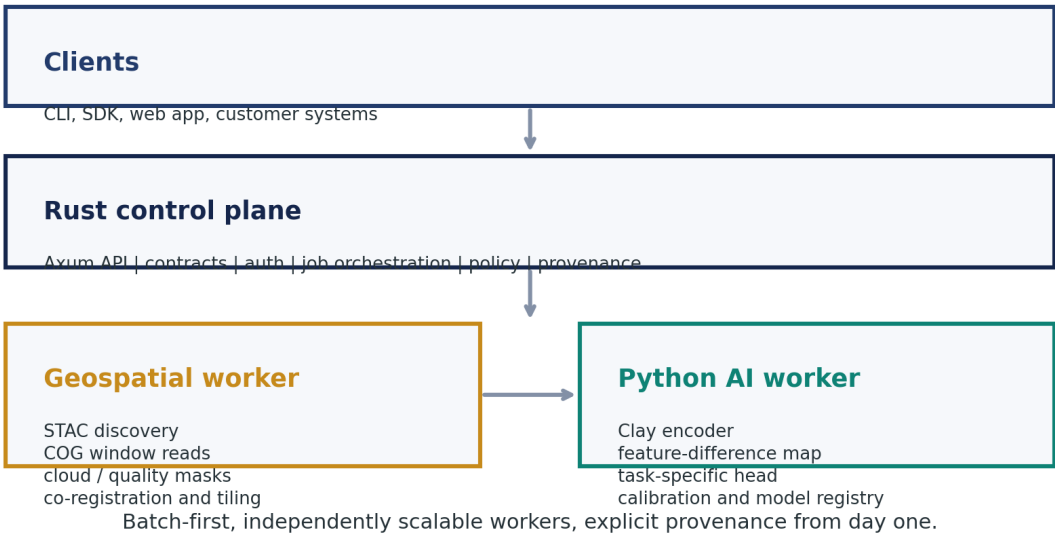


Figure 4. Rust owns the product control plane; Python owns the first geospatial and model runtime.

Recommended MVP components

Component	Initial choice	Why
API and control plane	Rust + Axum	Typed contracts, concurrency, job orchestration and strong systems signal
Geospatial processing	Python + Rasterio/GDAL	Mature ecosystem and lower integration risk
AI inference	Python + PyTorch + Clay	Matches the supported model ecosystem
Catalog	STAC adapters	Standardized scene discovery
Raster access	COG range reads	Avoids full-scene downloads
Metadata and jobs	SQLite first, Postgres after concurrency	Simple start with a clear migration path
Artifacts	Local disk or S3-compatible object store	Before/after images, overlays and exported rasters
Packaging	Docker Compose	Honest dependency boundary and reproducible startup

Data handling rules

- Use Sentinel-2 Level-2A surface reflectance first. Do not build atmospheric correction into the MVP.
- Use date windows rather than exact dates; observation availability depends on revisit and cloud conditions.
- Rank local valid-pixel quality over the area of interest, not only scene-level cloud percentage.
- Mask cloud, shadow, snow and invalid pixels using available quality bands.
- Read and cache analysis windows and prepared tiles, not arbitrary full satellite products.
- Use 256 x 256, 10-band chips for the Clay path unless experiments justify a different model contract.

Scaling stages

Stage	Architecture	Trigger to advance
Prototype	Local Python benchmark, occasional GPU pod	The complete data-to-model path works
Pilot	Rust API + one queue + ephemeral GPU worker + object storage	Two customers run repeated jobs
Hosted beta	Postgres, durable queue, autoscaled workers, observability and quotas	Recurring demand and measured utilization
Growth	Multiple GPU pools, model registry, tenant	Tenants have different workloads and SLAs

	isolation and lifecycle policies	
Enterprise	Private networking, dedicated deployments, audit logs and signed offline bundles	A funded customer requirement exists

Operational design principles

- Use asynchronous jobs, idempotency keys, retries, checksums and explicit job states from the beginning.
- Separate the control plane, raster work and model inference so each scales independently.
- Use ephemeral GPUs while utilization is low; reserve capacity only after measured throughput justifies it.
- Treat provenance as product data, not logging: every finding needs source, model and configuration identity.
- Keep telemetry disabled by default for self-hosted deployment and document all external data flows.

Reality check

The Rust API is not the expensive part. The hard engineering lies in scene quality, registration, evaluation and reliable operation across imperfect observations.

7. Competitive Positioning and Defensibility

The market is not empty. Oberon must own a narrower workflow.

Alternative	What it does well	How Oberon must differ
Lens by Upstream Tech	Turnkey monitoring, broad data, analyst services and conservation trust	API-first deployment, inspectable pipeline, customer-controlled models and workflow integration
LGND	Low-friction geospatial embeddings and developer APIs	Own an end-to-end operational change workflow, not only embeddings
Picterra	Custom enterprise GeoAI and environmental intelligence	Simpler deployable runtime, public benchmarks and transparent task contracts
Planet / SkyFi	Imagery access, commercial resolution and procurement	Remain data-source agnostic and treat imagery vendors as inputs
eoAPI / TiTiler / openEO	Open discovery, serving and processing infrastructure	Add task evaluation, evidence, provenance, review states and operating workflow
Raster Vision	Open-source geospatial ML framework	Provide a ready-to-query product runtime
In-house GIS scripts	Flexible and familiar to analysts	Reduce repeated plumbing and standardize quality, outputs and deployment

Positioning statement

External positioning

Oberon is the open, deployable inference and evidence runtime for teams that need to turn public Earth observation data into repeatable change findings inside their own systems.

What can become a moat

Potential moat	How it is earned	Why it matters
Task quality	Evaluated models, thresholds and abstention across regions and seasons	Customers trust outcomes, not a generic model call
Workflow reliability	Scene ranking, alignment, quality handling and provenance	The integration layer becomes hard to replace operationally
Feedback data	Customer-approved labels and error reviews with clear ownership	Supports calibration and domain-specific improvement
Distribution	The easiest open runtime for deploying geospatial task models	Partners and developers bring new use cases
Integration	Adapters for catalogs, sensors, downstream systems and reporting	Raises switching cost without locking data
Operational trust	Transparent failure behavior, versioning and support	Creates enterprise willingness to depend on the system

What is not a moat

- Calling Clay from an API.
- Writing the control plane in Rust.
- Using public Sentinel-2 imagery.
- Being first to publish a generic change endpoint.
- GitHub stars without repeat deployments or integrations.

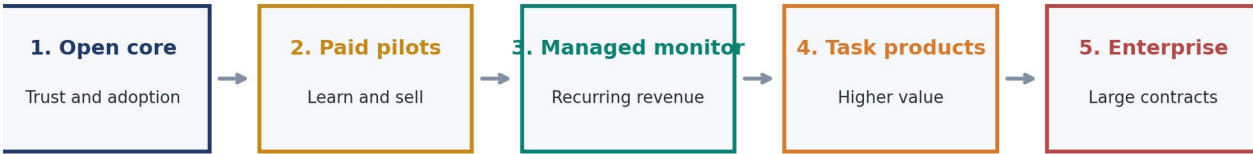
Defensibility principle

The implementation is replicable. Durable value comes from evaluated task quality, operational reliability, customer workflow integration and trusted feedback data.

8. Open Source to Commercial Product

Open source is the distribution and trust strategy, not the revenue model.

How Oberon grows from open source into a company



Each step requires proof before the next layer is built.

Figure 5. The company grows only after each evidence gate is crossed.

Recommended license

Use Apache 2.0 for Oberon Core. The permissive license supports adoption, commercial integration and enterprise comfort, while preserving a patent grant. The risk of a hosted fork is real, but the more immediate risk is no adoption. Protect the Oberon name through a trademark policy and define contribution ownership through a Developer Certificate of Origin or contributor agreement.

Product and code boundary

Offering	Access	What it contains	Why customers pay
Oberon Core	Apache 2.0	STAC/COG pipeline, Sentinel-2 preparation, baselines, general change workflow, API, CLI, evidence and provenance	They do not; it creates trust, usage and integrations
Reference task	Open source	One evaluated vegetation-change baseline, benchmark harness and documented limitations	Proves the architecture is real
Oberon Monitor	Commercial cloud	Managed compute, schedules, queues, storage, review workflow, alerts and collaboration	Avoids operating geospatial and GPU infrastructure
Oberon Tasks	Mixed / commercial	Maintained task models, thresholds, evaluation reports and supported regions	Tested outcomes, updates and accountability
Oberon Enterprise	Commercial	SSO, RBAC, audit, private registry, offline packaging, HA, SLA and support	Governance, procurement and reliability
Applied AI services	Commercial services	Data preparation, fine-tuning, calibration, integration and training	Custom needs and faster deployment

Repository structure

oberon-core/	# public: Rust API, workers, CLI, baseline task
oberon-model-adapters/	# public: Clay and future backbone interfaces
oberon-benchmarks/	# public: evaluation harness and examples
oberon-cloud/	# private: accounts, scheduling, billing, storage, alerts
oberon-enterprise/	# private: governance, deployment automation, offline bundles
customer-models/	# private per customer: labels, weights, thresholds, reports

Why this structure works

- The free product completes a real workflow, so developers can trust and adopt it.
- Paid value is not artificial restriction. It is operation, specialization, integration and accountability.
- Self-hosting supports data control and technical users, while managed cloud captures teams that do not want infrastructure burden.
- Enterprise work is built only after customers request and fund it.

Open-core rule

Never make the core an empty shell. If the open version cannot produce a complete evidence-backed change result, it will not generate trust or distribution.

9. Go-to-Market and Revenue Strategy

Sell a measurable monitoring outcome before selling platform access.

The first paid offer

Oberon Monitoring Pilot

A 4-8 week engagement that backtests one defined task on a customer portfolio, configures recurring analysis, measures analyst time and false-positive burden, and ends with a stop, extend or subscription decision.

Pilot element	Recommendation
Inputs	100-1,000 polygons, a historical window, the current review workflow and 20-50 reviewed examples
Deliverables	Baseline vs. AI benchmark, ranked findings, evidence artifacts, error review, deployment recommendation and integration plan
Success metrics	Useful findings per review hour, precision at K, false-positive burden, portfolio triaged and customer willingness to continue
Commercial outcome	Convert to managed monitoring, enterprise support or a funded customer-specific task model
Rejection criteria	No reviewer, undefined task, unsuitable resolution, regulated automation requirement, or no budget for required commercial imagery

Founder-led sales motion

1. Interview 15-20 technical and operational users before pitching platform features.
2. Ask for recent workflows, labor, deadlines, missed-change consequences, budgets and reviewed examples.
3. Build two design-partner benchmarks and one paid pilot proposal around a defined task.
4. Sell the outcome: faster, more auditable review. Do not sell Rust, Clay or "AI intelligence."
5. After two successful direct pilots, recruit environmental consultancies and geospatial integrators as channel partners.

Pricing hypotheses

Offer	Starting hypothesis	Notes
Open-source core	Free	Customer operates its own infrastructure
Research / nonprofit design pilot	\$0-\$5,000	Use selectively when learning, public evidence or credibility is unusually valuable
Commercial pilot	\$8,000-\$25,000	One task, historical benchmark and limited integration
Managed starter	\$250-\$750 / month	Small portfolio, batch cadence, public imagery and limited support
Managed team	\$1,000-\$3,000 / month	Larger portfolio, alerts, review queue, API, collaboration and reporting
Enterprise	\$25,000-\$100,000+ / year	Private deployment, governance, SLA, support and custom integrations
Custom task model	\$15,000-\$75,000+	Depends on labels, geography, evaluation and deployment

Pricing logic

Do not price by HTTP request. A request can represent one hectare or a region, two observations or dozens, and a cheap index or expensive model. Price using portfolio size, cadence, task complexity, retention, team workflow and deployment burden. Commercial imagery should be passed through or separately priced.

Internal cost = data access + CPU preprocessing + GPU seconds + artifact storage + retries + operations + support allocation

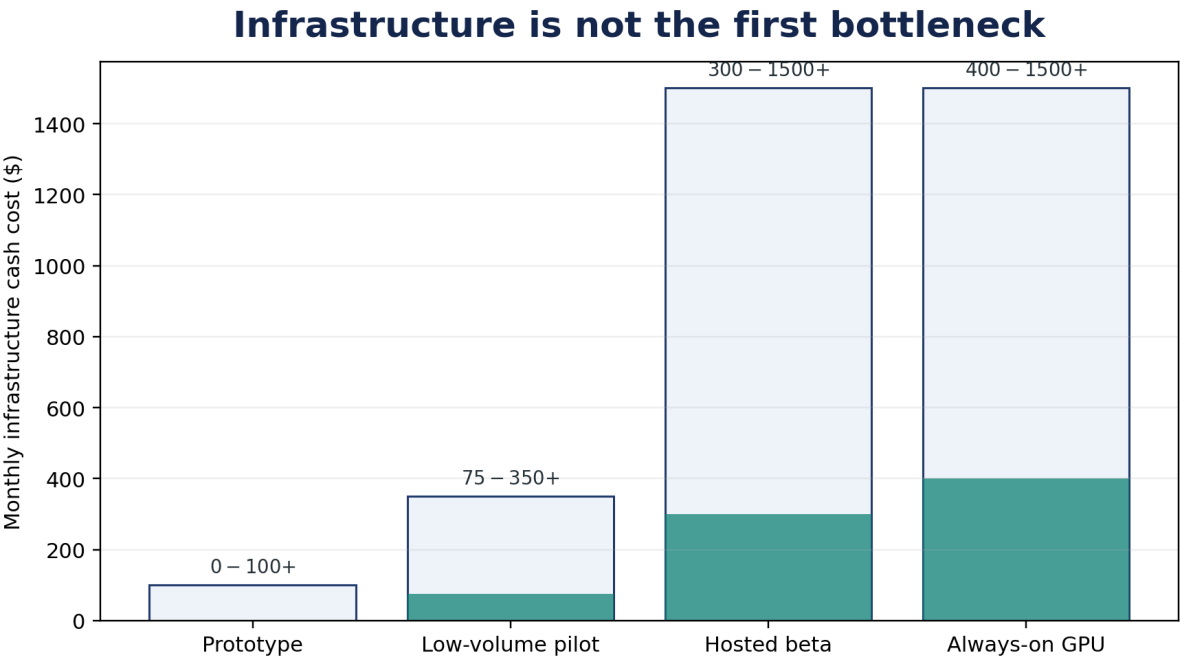
Price = the greater of cost-based margin or a justified share of customer value

Revenue principle

The customer buys fewer analyst hours, faster detection, better evidence and operational reliability. GPU minutes are an input cost, not the product value.

10. Economics and Operational Scale

The first constraints are trust, workflow quality and sales - not compute.



Support, model evaluation and sales will dominate early economics.

Figure 6. Illustrative infrastructure ranges before staff, sales and support.

Cost structure

Cost area	Early implication
Public imagery	Sentinel-2 and Landsat keep experimentation affordable, subject to provider access policies.
GPU inference	Batch and ephemeral use can be inexpensive; always-on GPUs create fixed cost without proving demand.
Object storage	Artifacts and cached chips remain manageable with retention and lifecycle rules.
CPU control plane	Usually not the cost driver. Reliability and operations matter more than raw compute.
Commercial imagery	Can exceed compute cost and should be passed through or separately quoted.
Human cost	Geospatial review, model evaluation, sales and support will dominate early gross margin.

Measurements required before self-serve pricing

- Chips processed per second by model, GPU and batch size.
- Data read volume and preprocessing time per square kilometer.
- Artifact size, retention and cache hit rate.
- Failure, retry and abstention rates.
- Analyst minutes saved per 100 monitored areas.
- Support hours per customer and per deployment profile.

Gross-margin discipline

A low inference bill does not guarantee a scalable business. If every customer needs custom GIS work, repeated threshold tuning and manual report assembly, the company is selling services regardless of software architecture. Productization means the support burden and analyst labor decline as customer count grows.

Signal	Healthy direction	Warning
Compute utilization	Higher batching and predictable scheduled jobs	Always-on capacity with low use
Support burden	Declines through documentation and standard workflows	Founder remains embedded in every analysis
Pilot similarity	Customers share the same task, inputs and output contract	Every pilot becomes a new product
Retention	Portfolios are monitored repeatedly	Users run one demo and disappear
Expansion	More areas, cadence, tasks or teams	Only discounts produce continued use

Scale decision

Move to dedicated GPU capacity only when measured utilization or latency requires it. Move to enterprise infrastructure only when a funded customer requires it.

11. Roadmap, Team and Capital

Each phase is an evidence gate, not a calendar promise.

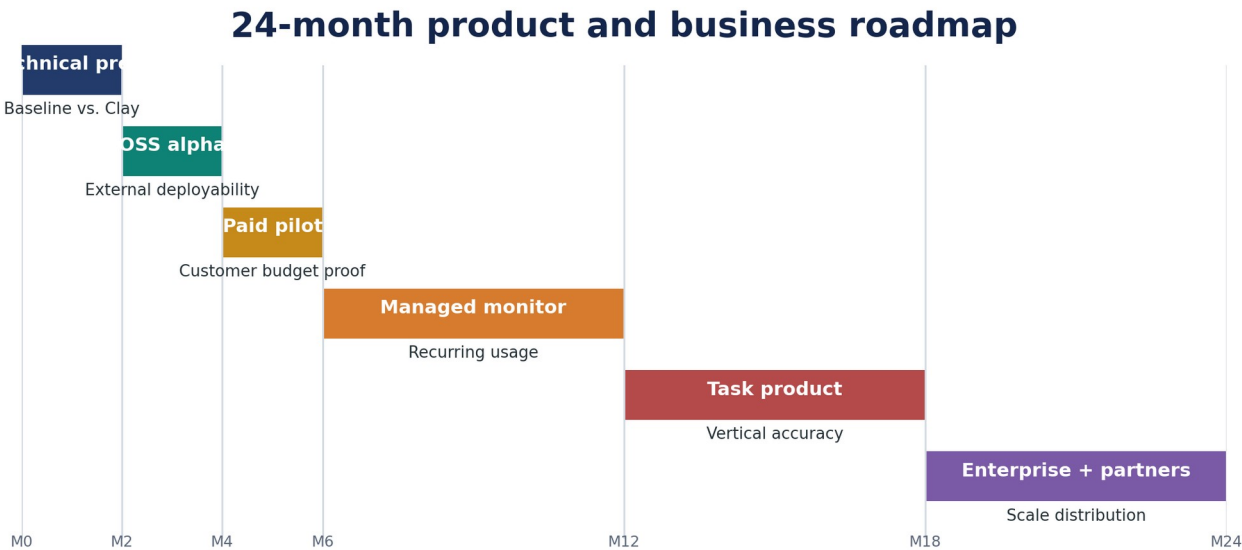


Figure 7. The recommended sequence from technical proof to enterprise scale.

Milestones and business gates

Period	Product milestone	Market milestone	Gate
Month 0-2	Python benchmark, one task, one dataset and evidence artifacts	15 interviews; 3 organizations provide examples	Clay beats or complements a simple baseline
Month 2-4	OSS alpha, Docker Compose, API and reproducible benchmark	2 design partners and first technical launch	External user deploys without live help
Month 4-6	Review workflow, scheduled batch job and task registry	1 paid pilot and 1 case study	Customer pays to continue or integrate
Month 6-12	Hosted beta, alerting, portfolio management and vegetation task v1	3-5 paid pilots, 2 recurring customers and first consultancy partner	One segment and task repeat
Month 12-18	Multi-tenant cloud, model registry and commercial imagery adapter	5-15 recurring customers or 2-4 enterprise/partner contracts	Gross margin and support burden are measurable
Month 18-24	Enterprise deployment pack, second task and partner toolkit	Renewals and repeatable outbound/channel motion	Choose focused profitable path or venture expansion

Hiring triggers

Trigger	Hire or contractor	Reason
Two simultaneous pilots	Part-time geospatial scientist	Evaluation, labels, quality review and customer credibility
Hosted beta has repeated jobs	Platform / MLOps engineer	Queues, observability, GPU utilization, reliability and security
Three recurring customers in one vertical	Domain product lead or solutions engineer	Turn buyer workflow into a repeatable product
Founder cannot support pipeline and sell	Customer success / solutions hire	Protect founder time for product and sales
Funded enterprise requirement	Security and deployment expertise	Do not pre-hire for speculative compliance

Funding sequence

6. Founder-funded technical proof using local and short-lived GPU resources.
7. Paid pilots that fund customer-specific learning.
8. Small grants or research partnerships only when the solicitation directly matches the product task.
9. Revenue-financed hosted beta after repeated usage.
10. Angel or seed capital only after a repeatable task, paying customers and evidence that capital accelerates a working motion.

Capital discipline

Raising before customer and task proof risks forcing Oberon back into a broad platform story. Capital becomes rational when the bottleneck is execution scale, not product discovery.

12. Risks, Decision Gates and Probability Scenarios

The project remains valuable even if the business case fails.

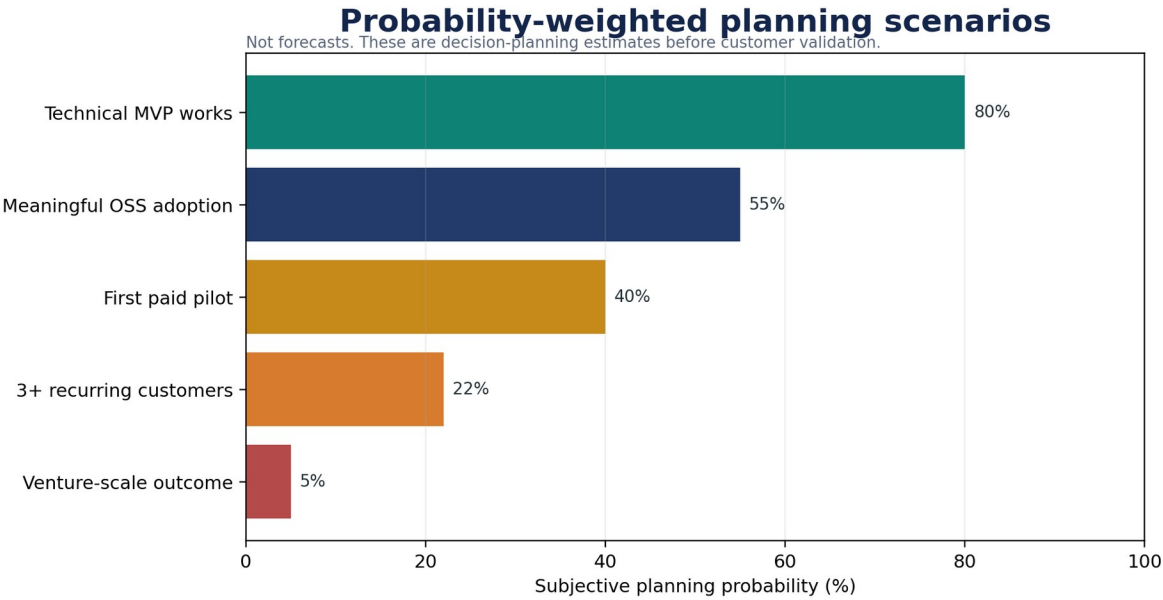


Figure 8. Subjective planning probabilities before customer validation.

Primary risks

Risk	Why it can kill the business	Response
AI adds no value	Simple indices may solve the first task	Benchmark openly and remove Clay from the default if it does not win
Seasonality creates false change	Different dates look different without operational events	Season matching, composites, quality masks and abstention
Self-hosting is not valued	Most buyers may prefer managed products	Use self-hosting for trust; monetize managed operation
Sentinel-2 is too coarse	Small objects and parcel detail are not resolvable	State limits and add premium imagery adapters after demand

Incumbents already solve the job	Lens and vertical vendors have trust and distribution	Target integration, openness and underserved technical teams
Pilots become custom consulting	No repeatable product emerges	Narrow the vertical or choose a services business intentionally
Enterprise procurement drags	Long cycles consume founder capacity	Start with consultancies and restoration teams
Founder scope overload	Rust, geospatial, ML, OSS, sales and support compete	Use strict gates, Python-first proofs and contractors

Hard decision gates

Gate	Continue when	Pivot or stop when
Technical - Week 4	AI improves ranking or complements baselines on held-out examples	Clay adds no useful signal and another task is not clearly better
Deployment - Week 8	An external user reproduces the benchmark and runs the service	Packaging remains founder-dependent
User - 15-20 interviews	A repeated monitoring workflow, reviewed examples and pain are visible	Interest is occasional and no workflow owner exists
Commercial - after credible demo	At least one buyer pays for a scoped pilot	No one funds the evaluation despite a strong demo
Product - after pilots	Customers share inputs, outputs and task definition	Every pilot is unrelated custom work
Enterprise - before compliance spend	A real buyer requests controls and has budget	Security work remains hypothetical

Probability-weighted outcomes

The most likely successful outcome is not a global satellite platform. It is either a respected open-source project with strong career value, or a focused monitoring and applied AI business serving a narrow vertical. Venture-scale potential exists only after a repeated task produces retained usage, proprietary evaluation data and partner distribution.

Outcome	Planning probability	Implication
Technically strong open-source project	High	Worth building even if commercialization stops
Paid pilot and services revenue	Medium	Plausible with direct discovery and a credible benchmark
Sustainable focused software business	Low-medium	Requires repeatable task, recurring customers and declining support burden
Venture-scale platform	Low	Requires a large vertical, channel leverage, data moat and multiple task products

Founder safeguard

If the commercial gates fail, stop adding business overhead. Preserve Oberon as a high-quality open-source systems and AI portfolio project.

13. External Product and Investor Pitch

A concise narrative that does not overclaim.

The problem

Public Earth observation data is abundant, but operational monitoring remains expensive and fragmented. Teams still spend analyst time finding comparable observations, cleaning imagery, reviewing false changes and assembling evidence. Existing infrastructure exposes data and models, but the last-mile decision workflow remains difficult to deploy, audit and integrate.

The product

Oberon is an open, self-hostable change-intelligence engine. It automatically selects suitable before/after observations, prepares comparable multispectral imagery, combines interpretable spectral measurements with geospatial AI, ranks likely material changes, and returns review-ready geometry, artifacts, quality states and provenance.

The initial market

The first wedge is vegetation disturbance and restoration monitoring for environmental consultancies, project developers and conservation teams managing repeated land portfolios. These users already spend money on remote monitoring, analyst work and field verification.

Why now

- STAC and COG make public imagery increasingly accessible and cloud-native.
- Sentinel-2 Level-2A offers analysis-ready global multispectral coverage.
- Open geospatial foundation models create a reusable representation layer.
- Compliance, restoration and digital MRV are increasing demand for auditable monitoring.
- Infrastructure cost is low enough that workflow quality and trust become the primary differentiators.

Business model

Apache-licensed core creates trust and technical distribution. Revenue comes from paid monitoring pilots, managed portfolio operation, evaluated task products, custom model work and enterprise deployment controls.

Why Oberon can win

Oberon does not need to own satellites or outspend existing platforms. It must become the easiest way for technical teams to deploy a transparent, evidence-backed monitoring workflow inside their own systems. The moat, if earned, will be task quality, workflow reliability, customer feedback data and partner integration.

One-line pitch

Oberon turns public satellite observations into ranked, verifiable land-change findings - open enough to trust, operational enough to depend on.

The next 30 days

Week	Build	Market	Decision
1	Create 10-20 paired Sentinel-2 examples; compare pixel, NDVI and Clay change	Build a 30-account list; schedule five interviews	Does AI show useful signal?
2	Return before, after, overlay, quality, provenance and ranked polygons	Interview five more users; obtain two historical examples	Is there a repeated job?
3	Package a Docker demo and benchmark methodology	Offer two design-partner benchmarks and one paid proposal	Will anyone commit data, time or money?
4	Fix deployment and write a concise technical README	Choose one ICP and task	Continue, pivot task or remain OSS

A. Sources and Validation Notes

Current public sources and internal documents used to shape the strategy.

Sources were reviewed in the supplied research through June 19, 2026. Prices, product capabilities and regulatory dates can change and should be revalidated before external publication or commercial quoting. Named organizations are prospect examples, not confirmed leads.

- [1] Oberon Product Brief v3 - AI-Centered, internal, June 19, 2026.
- [2] Oberon Market Strategy and Scale Plan v1, internal, June 19, 2026.
- [3] STAC specification - <https://stacspect.org/>
- [4] eoAPI documentation - <https://eoapi.dev/intro/>
- [5] TiTiler documentation - <https://developmentseed.org/titiler/>
- [6] openEO documentation - <https://openeo.org/documentation/1.0/>
- [7] Raster Vision documentation - <https://docs.rastervision.io/>
- [8] Copernicus Data Space - Sentinel-2 documentation - <https://documentation.dataspace.copernicus.eu/Data/SentinelMissions/Sentinel2.html>
- [9] AWS Open Data - Sentinel-2 L2A COGs - <https://registry.opendata.aws/sentinel-2-l2a-cogs/>
- [10] Clay Foundation Model - overview - <https://clay-foundation.github.io/model/>
- [11] Clay basic use and Sentinel-2 input - https://clay-foundation.github.io/model/getting-started/basic_use.html
- [12] Clay model specification and release notes - <https://clay-foundation.github.io/model/release-notes/specification.html>
- [13] Lens by Upstream Tech - <https://lens.tech/>
- [14] Lens plans and data pricing - <https://lens.tech/plans/>
- [15] LGND pricing - <https://lgnd.ai/pricing>

[16] Picterra - <https://picterra.ai/>

[17] SkyFi pricing - <https://skyfi.com/en/pricing>

[18] European Commission - EU deforestation regulation - https://environment.ec.europa.eu/topics/forests/deforestation/regulation-deforestation-free-products_en

[19] Verra - digital forest carbon MRV - <https://verra.org/worlds-largest-carbon-program-pilots-digital-measuring-of-forest-carbon/>

[20] Apache License 2.0 - <https://www.apache.org/licenses/LICENSE-2.0>

[21] GitLab open-core stewardship - <https://handbook.gitlab.com/handbook/company/stewardship/>

[22] Supabase self-hosting guidance - <https://supabase.com/docs/guides/self-hosting>

[23] RunPod GPU pricing - <https://www.runpod.io/pricing>

[24] Cloudflare R2 pricing - <https://developers.cloudflare.com/r2/pricing/>

[25] Osa Conservation research and collaboration - <https://osaconservation.org/database/>

[26] Terraformation / Terraware - <https://info.terraformation.com/terraware>

[27] Guo et al., On Calibration of Modern Neural Networks - <https://arxiv.org/abs/1706.04599>

Validation labels used in this report

Label	Meaning
Supported	Primary documentation or direct market evidence supports the statement.
Partially validated	Adjacent tools and buyer behavior support the thesis, but Oberon-specific proof is missing.
Unvalidated	Requires customer interviews, usage, pilots or payment evidence.
Planning estimate	A scenario, cost range, price or probability used for decisions rather than a forecast.

Final recommendation

Build the smallest system in which AI measurably reduces human review. Prove one repeated workflow, one paying pilot and one independently deployable open-source release before expanding Oberon into a platform.